

TEMA 1 – Hojas de Estilo

🎯 Objetivo del tema

En este tema profundizaremos en las Hojas de Estilo CSS

1.1 Introducción

En general, los servidores responden al Cliente (navegador web) enviándole de vuelta ficheros de **texto plano** que, todos ellos juntos, conforman lo que normalmente llamamos “la página web”.

Estos ficheros de texto plano son:

- **HTML (Estructura y Semántica):** Define el árbol de contenido (DOM) y qué elementos existen: botones, tablas, formularios...
- **CSS (Presentación y Diseño Adaptativo):** Define cómo se ven esos elementos: maquetación, colores, tipografía y la adaptación a distintas pantallas (*Responsive Web Design*).
- **JavaScript (Comportamiento y Lógica):** Nos permite modificar el HTML y el CSS en tiempo real, procesa datos, capturar la interacción del usuario y se comunica con el servidor en segundo plano.

Lo que nos interesa en este módulo es el CSS, las Hojas de Estilo en Cascada. Ya hemos visto en la UD anterior que el aspecto visual de una web es mucho más que simplemente hacer una página bonita.

La idea detrás del CSS fue (originalmente y hace décadas) tener separado el contenido de la página web de su aspecto visual. De esta forma, podemos por ejemplo definir varias hojas de estilos para un mismo sitio web, y permitir al usuario alternar entre ellas a su gusto. Y por supuesto, facilita el mantenimiento, mostrar la página web en diferentes dispositivos, etc.

Pero todo esto ya lo sabes.

1.2 Historia del CSS

No existe un CSS único. Hay varios **niveles** diferentes, contruidos uno sobre el otro. En esto es similar a las versiones de software: una versión superior incluye todo lo de las versiones anteriores, más una serie de funcionalidades nuevas.

Niveles de CSS.

- **CSS 1:** Fue la primera especificación oficial de CSS recomendada por la W3C, publicada en diciembre 1996, y abandonada en abril de 2008.
- **CSS 2:** Fue desarrollada por la W3C y publicada como recomendación en mayo de 1998, Abandonada en abril de 2008. Ofrece como añadido funcionalidades de las capas <div>, como posicionamiento relativo/absoluto/fijo, etc.
- **CSS 2.1:** La primera revisión de CSS2, usualmente conocida como "CSS 2.1", corrige algunos errores encontrados en CSS2, elimina funcionalidades poco soportadas o inoperables en los navegadores y añade alguna nueva especificación.
- **CSS 3:** La versión actual, publicada en 2011. A diferencia de CSS2, CSS3 está dividida en varios documentos separados, llamados **módulos**. Cada módulo añade nuevas funcionalidades a las definidas en CSS2, pero **preservando** las anteriores para mantener la compatibilidad
- **CSS 4:** No existe ni existirá nunca. El W3C no va a publicar nunca un paquete oficial llamado "CSS4". La idea de lanzar grandes versiones globales (como ocurrió con CSS1 y CSS2) se abandonó por complete. El futuro del CSS serán los **módulos independientes** que se irán añadiendo sobre el CSS3.

Cuando hablamos de **módulos**, tenemos que imaginarnos que son 'piezas independientes' del lenguaje CSS. Puedes pensar en ellos como las librerías de Java: las añades y te aportan funcionalidades nuevas a tu web. Un ejemplo de un módulo añadido posteriormente sobre CSS 3 fue **CSS View Transitions Module (Nivel 1)**, un módulo completamente nuevo diseñado para crear transiciones fluidas y animadas entre diferentes páginas.

No tenemos que hacer nada especial para usar estos **módulos** en nuestros diseños. Los navegadores web se actualizan regularmente de forma automática para ser capaces de interpretar estos módulos.

1.3 Soporte del CSS en navegadores

Internamente los navegadores están divididos en varios componentes. La parte del navegador que se encarga de interpretar el código HTML y CSS para mostrar las páginas se denomina **motor**. Desde el punto de vista del diseñador CSS, la versión de un motor es mucho más importante que la versión del propio navegador.

Los principales navegadores modernos (Chrome, Edge, Firefox y Safari) ofrecen un soporte completo para los estándares modernos de CSS (incluyendo Flexbox, Grid, Container Queries y variables avanzadas), lo que garantiza una compatibilidad casi idéntica en el motor de renderizado de la web.

Pero esto no fue siempre así...

Internet Explorer

Internet Explorer es un navegador oficialmente discontinuado y en desuso. Históricamente presentó graves carencias y errores en su soporte de CSS hasta su versión 8. La versión 6 (IE6) del 2001 se convirtió en la pesadilla de los desarrolladores web porque combinó un dominio casi absoluto del mercado con un estancamiento técnico de más de cinco años sin actualizaciones significativas. Por poner un ejemplo:

- IE6 interpretaba el cálculo de los anchos y márgenes en CSS de forma diferente al estándar oficial del W3C. Todo el mundo hacía `width: 200px + padding: 10px = 220px`. Excepto IE6... que decía que eran 200px. Esto rompía las maquetaciones.
- No soportaba las transparencias en los PNG
- Carecía de soporte para selecciones de CSS2, como `:first-child`.
- Para que un sitio web se viera igual en Firefox/Opera/Safari y en IE6, los desarrolladores tenían que escribir código duplicado o "hacer trucos" específicos para que funcionase.

Pero lo que nos interesa: **Los estándares y recomendaciones existen por un motivo**. Y nadie quiere que se repita una 'Guerra de navegadores'. Es por este motivo que vamos a intentar ajustarnos lo máximo posible a las diferentes normativas.

Importante – La Guerra de Navegadores

Es interesante que consultes en Internet lo que fue la llamada "Guerra de Navegadores". Ocurrió en etapas a lo largo de los años, y ha influenciado lo que es hoy en día la web. Comprender lo que motivó esta 'guerra' y los motivos de cada una de las partes, te ayudará a comprender por qué hacemos las cosas de una forma y no de otra hoy en día.

1.4 CSS y maquetación

Antes de la aparición de la primera versión de CSS, el concepto de **maquetar** una página web (darle estilos) era muy diferente al que tenemos actualmente. En aquella época se usaban exclusivamente etiquetas HTML para darles algo de formato a las páginas.

Por ejemplo:

```
<body>
  <h1><font color="red" face="Arial" size="5">Titular de la página</font></h1>
  <p><font color="gray" face="Verdana" size="2">Un párrafo de texto no muy largo.</font></p>
</body>
```

Esto nos mostraba:

Titular de la página

Un párrafo de texto no muy largo.

Obviamente, las opciones eran relativamente limitadas. No obstante, Internet creció y pronto se dieron cuenta de el problema: manejar muchos estilos diferentes se vuelve **complicado**.

Solución:

El CSS (Hojas de Estilos en Cascada). Agrupamos los estilos al principio del documento, separando así el **qué** tenemos que mostrar del **cómo** se muestra.

Los estilos se colocaban en la etiqueta <head> del HTML.

```
<style type="text/css">
  h1 { color: red; font-family: Arial; font-size: large; }
  P { color: gray; font-family: Verdana; font-size: medium; }
</style>
```

Lo cuál fue, en el momento, una súper idea.

Hasta que, de nuevo, los estilos se volvieron demasiado **complicados** como para manejarlos de esta manera.

Solución:

Dejar a los programadores que se organicen como quieran. Se permitió que los estilos se colocasen en **tres** **sitios** diferentes, que cualquiera de las opciones fuese 'la buena', y que incluso pudieses 'mezclar' cosas y la web funcionase.

Los tres sitios son:

- Poner el CSS en el propio componente HTML (la etiqueta).

Ya lo hemos visto más arriba. Desaconsejada. Date cuenta que estás maquetando con HTML. ¿Quizás para un arreglo puntual?

- Poner el CSS en la <head> del HTML.

Desaconsejada. Las reglas son sencillas: En la <head> del documento se añade una etiqueta <style> con los estilos deseados. El formato es el clásico: poner { } y los formatos a aplicar. Se usa **ocasionalmente** cuando una página web en concreto tiene que tener un par de variaciones respecto a los estilos generales del resto del sitio web, los cuales se indican en un fichero aparte.

- Poner el CSS en un fichero aparte.

Esta es la idea estándar por defecto. La opción de libro. Incluir en el <head> un <link> al archivo CSS. Esto permite reutilizar los mismos estilos para toda una colección de páginas web. Facilita el mantenimiento, y permite dar a un sitio web un aspecto homogéneo.

El problema ocurre cuando quieres mezclar las tres formas anteriores a la vez. Cuando aplicas estilos a un mismo elemento usando distintos métodos, gana el que tenga mayor **especificidad**:

1. Usar un atributo en línea (**style=""**) tiene la **Máxima prioridad**. Es decir, que se superpone a los estilos indicados en el <head> y en el archivo .css.
2. Los estilos del **<head>** o los de un **fichero externo** tienen la misma prioridad base. Si compiten por la misma propiedad, gana el que esté escrito **más abajo** en el HTML.
3. **!important** es una regla especial que puedes añadir a cualquier propiedad para forzar que gane, sin importar dónde esté escrita. Mala idea abusar de ella.

1.5 Incluir desde archivo externo

Cuando un navegador carga una página HTML, simplificando las cosas, le llegan tres ficheros: El HTML, el CSS y el JavaScript. Esto lo hace por partes. Primero el HTML, y luego todos los ficheros enlazados.

Si decidimos usar **<Link>** para enlazar los CSS, deberemos usar sus cuatro atributos:

- **rel**: indica el tipo de relación que existe entre el recurso enlazado (en este caso, el archivo CSS) y la página HTML. Para los CSS, siempre se utiliza el valor **stylesheet**
- **type**: indica el tipo de recurso enlazado. Sus valores están estandarizados y para los archivos CSS su valor siempre es **text/css**

- **href:** indica la URL del archivo CSS que contiene los estilos. La URL indicada puede ser relativa o absoluta y puede apuntar a un recurso interno o externo al sitio web.

Por favor, **no** uséis rutas absolutas...

- **media:** indica el medio en el que se van a aplicar los estilos del archivo CSS. Lo veremos más adelante. Por ahora, podemos ponerle **screen**.

A modo de ejemplo:

```
<link rel="stylesheet" type="text/css" href="/css/estilos.css" media="screen">
```

También podemos usar **<style>** para enlazar un fichero. En este caso se utiliza una regla especial **@import**. Las reglas de tipo **@import** siempre preceden a cualquier otra regla CSS (con la única excepción de la regla **@charset**). La URL del archivo CSS externo se indica mediante una cadena de texto con comillas simples, dobles o mediante la palabra reservada **url**.

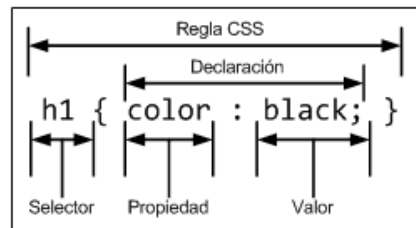
```
@import '/css/estilos.css'  
@import "/css/estilos.css"  
@import url('/css/estilos.css'  
@import url("/css/estilos.css")
```

A modo de ejemplo:

```
<style type="text/css" media="screen">  
    @import '/css/estilos.css'  
</style>
```

1.6 Añadir un estilo

Para añadir un estilo, tenemos que seguir unas **reglas** estrictas. Una regla es un “estilo individual” aplicado a un elemento del HTML. Podemos escribir tantas reglas como queramos y aplicarlas sobre cualquier elemento que necesitemos. Como ya comentamos antes, el problema ocurre cuando varias reglas hacen referencia al mismo elemento.



- **Selector**: indica el elemento o elementos HTML a los que se aplica la regla CSS.
- **Propiedad**: lo que queremos modificar como, por ejemplo: color, tamaño, borde...
- **Valor**: el nuevo valor que va a tener esa propiedad de ese selector.

🔔 Importante – ¿Me tengo que saber todas las propiedades?

No. El estándar CSS 2.1 en su momento definía 115 propiedades cada una con su propia lista de valores permitidos. Los últimos borradores de CSS 3 ya incluían 239 propiedades para el año 2011. Hoy en día, CSS3 ya no es un bloque monolítico único, está formado por componentes independientes.

Aprende a consultar las cosas. Tienes la **W3CSchools** y las herramientas de **DevTools**. Y cuando usemos un IDE en condiciones, verás que te sirven de ayuda.

Selectores

Un **selector** permite seleccionar a qué elemento concreto del HTML se le aplica una regla. Hay muchos selectores, y todos ellos nos permiten seleccionar diferentes cosas.

- **Selector universal *** - Seleccionar **todos** los elementos de la página. Se usa para dar los estilos más generales a una página. A modo de ejemplo, para eliminar el margen y el relleno de todos los elementos HTML haríamos:

```
* {  
    margin: 0;  
    padding: 0;  
}
```

- **Selector de tipo o de etiqueta** – Selecciona **todos** los elementos de la página cuya etiqueta HTML coincide con el valor del selector. Para dar un estilo a todos los párrafos:

```
p {
    color: black;
}
```

Es posible agrupar todos los selectores para aplicar estilos a un varios elementos:

```
/* Todos los H1, H2 y H3 iguales*/
h1, h2, h3 {
    color: black;
}

/* Truco para agrupar estilos */
h1, h2, h3 {
    color: black;
}

h1 {font-size: 2em;}
h2 {font-size: 1.5em;}
h3 {font-size: 1.2em;}
```

- **Selector descendente** – O, dicho de otra forma, “seleccionar al Hijo”. Es posible seleccionar un elemento HTML que está dentro de otro HTML. Decimos que un elemento es “Hijo” de otro elemento cuando se encuentra entre < y /> de otro HTML.

Si hacemos:

```
p span { color: red; }
h1 span { color: blue;}
```

Lo que obtendremos es:

- Los elementos que se encuentran dentro de un elemento <p> se muestran de color rojo.
- Los elementos que se encuentran dentro de un elemento <h1> se muestran de azul.
- El resto de elementos de la página se ignoran.

Nótese que mientras haya un dentro de <p>, no importa dónde, se pondrá rojo. Es decir: que este selector afectaría a **todos** los elementos “descendientes”, ya sean hijos, nietos, bisnietos...

```
p a span em { text decoration : underline ;}
```

De la misma forma, esto afectaría a **todos** los que se encuentren dentro de elementos de tipo , que a su vez se encuentren dentro de elementos de tipo <a>, que se encuentren dentro de elementos de tipo <p>.

- **Selector de Clase** – Se emplean para responder a la necesidad de aplicar un estilo a un único elemento concreto. ¿Cómo se pueden aplicar estilos CSS sólo al primer párrafo?

La solución más simple es utilizar el atributo **class** de HTML. Una regla así creada se aplica sobre **todos** los elementos que tengan el mismo nombre en su atributo class.

Fíjate que usamos un (.) para identificar este tipo de selectores.

```
<body>
|   <p class="myEstilo">Lorem ipsum dolor sit amet... </p>
| </body>

/* Se aplica a todos los class="myEstilo" */
.myEstilo {color: black;}
```

Debido a cómo “funciona” CSS, es posible incluso ser más detallista:

```
p.myEstilo {color: black;}
```

Se aplicaría a todos aquellos elementos de tipo <p> que tengas como atributo class “myEstilo”. Por tanto, el selector de clase suele emplearse para destacar casos particulares, avisos, errores, etc. Obviamente, es imprescindible utilizarlos para crear webs complejas con muchos componentes y estilos diferentes.

Y ahora... ¡intenta no confundirte!

```
/* Todos los elementos de tipo "p" con atributo class ="myEstilo" */
p.myEstilo {color: black;}

/* Todos los elementos con atributo class ="myEstilo" que estén dentro de "p" */
p .myEstilo {color: black;}

/* Todos los elementos con atributo class ="myEstilo" Y todos los "p" */
p, .myEstilo {color: black;}
```

Adicionalmente, es posible definir elementos con más de un atributo class. Por ejemplo, podemos aplicar los estilos definidos en estas 3 reglas de la siguiente forma:

```
<body>
|   <p class="myEstilo destacado error">Lorem ipsum dolor sit amet... </p>
| </body>
```

De la misma forma podemos definir estilos directamente a elementos que tengan varias clases. Este estilo se aplicaría a **todos** los que tengan los atributos class myEstilo y error:

```
.myEstilo.error {color: red;}
```

- **Selector de ID** – Similar al Selector de Clase, pero mediante la id del elemento HTML. Dado que las ID son únicas para una misma página, podemos alcanzar un nivel de detalle mucho mayor. Se emplea la almohadilla (#) como selector.

```
<body>
  <p id="myId">Lorem ipsum dolor sit amet... </p>
</body>
```

```
/* Sólo el componente id ="myId" */
#myId {color: black;}
```

¿Atributo class o Id? En realidad, no entran en conflicto. El selector de clase permite que varios elementos tengan el mismo estilo, pero el selector de id no. La recomendación general es la de utilizar el selector de ID cuando se quiere aplicar un estilo a un solo elemento específico de la página y utilizar el selector de clase cuando se quiere aplicar un estilo a varios elementos.

Al igual que los selectores de clase, en este caso también se puede restringir el alcance del selector.

```
/* Sólo el componente de tipo "p" con id ="myId" */
p#myId {color: black;}
```

Y ahora... ¡intenta no confundirte!

```
/* Sólo el componente de tipo "p" con id ="myId" */
p#myId {color: black;}

/* Todos los elementos id ="myId" que estén dentro de "p" */
p #myId {color: black;}

/* Todos los elementos id ="myId" Y todos los "p" */
p, #myId {color: black;}
```

Combinando Selectores

Es posible realizar combinaciones realmente extrañas de selectores. Aquí tienes varios ejemplos. Mira primero el selector y después intenta adivinar lo que significan... ^_^

```
.aviso .especial { ... }
```

Respuesta: Todos los elementos con un class="especial" que se encuentren dentro (espacio en blanco) de cualquier elemento con un class="aviso".

```
div .aviso span .especial { ... }
```

Respuesta: Todos los elementos de tipo con un atributo class="especial" que estén dentro (espacio en blanco) de cualquier elemento de tipo <div> que tenga un atributo class="aviso".

```
ul #menuPrincipal li .destacado a #inicio { ... }
```

Respuesta: El elemento con id igual a inicio que se encuentra dentro (espacio)de un elemento de con un atributo class igual a destacado, que forma parte de una lista con un atributo id igual a menuPrincipal.

Dispones de una serie de **ficheros con código** sobre los que tendrás que ir realizando los ejercicios propuestos.

Para este ejercicio, extrae los ficheros correspondientes al Ejercicio 1.

Cambios que tienes que realizar:

- Todas las esquinas de la caja id = “principal” tienen que tener una curvatura de 20px.
- Crea secciones con artículos. Dale forma a la esquina superior izquierda de 20 px, a la esquina superior derecha 10px, a la esquina inferior derecha 30px y a la esquina inferior izquierda 50 px.
- Aplica sombras a nuestra caja id = “principal” con un color RGB (150,150,150) y una distancia horizontal y vertical de la sombra al elemento de 5px.
- Convierte esa sombra externa en una sombra interna para proveer un efecto de profundidad.

A la derecha tienes una imagen de cómo debería quedar la página.

